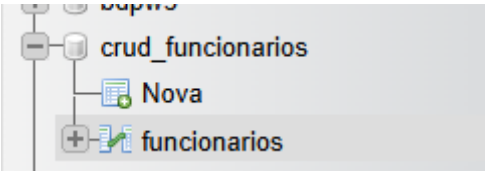
Aula 26 de maio de 2026

PROGRAMA DE COMPUTADORES

Banco de Dados




```
*SQL tab → Cole e Execute:**
```sql
CREATE DATABASE crud_funcionarios
USE crud_funcionarios;

CREATE TABLE funcionarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    funcao VARCHAR(100) NOT NULL,
    salario DECIMAL(10,2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

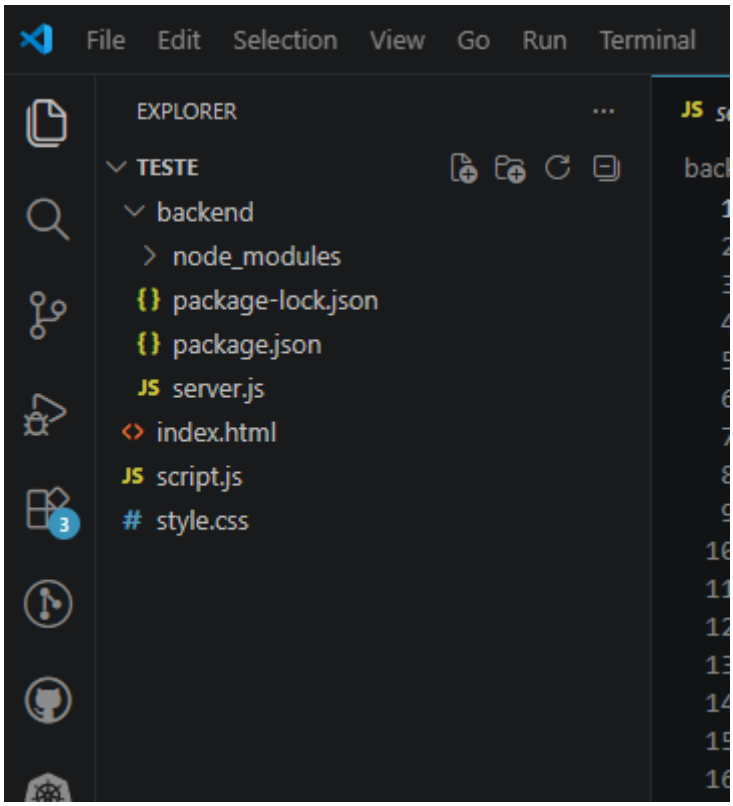
Esse código SQL é utilizado para criar a estrutura do banco de dados do sistema CRUD de funcionários. Ele deve ser executado na aba SQL de uma ferramenta de gerenciamento MySQL, como MySQL Workbench ou phpMyAdmin. O objetivo do script é criar um banco de dados e uma tabela responsável por armazenar as informações dos funcionários cadastrados pela aplicação. A primeira instrução, `CREATE DATABASE crud_funcionarios`, cria um novo banco de dados chamado `crud_funcionarios`, que servirá como local de armazenamento das informações do sistema. Em seguida, o comando `USE crud_funcionarios;` seleciona esse banco de dados para que os próximos comandos sejam executados dentro dele. Depois disso, o comando `CREATE TABLE funcionarios` cria uma tabela chamada `funcionarios`, que será utilizada para guardar os dados dos funcionários cadastrados.

Dentro da tabela são definidas várias colunas. A coluna `id INT AUTO_INCREMENT PRIMARY KEY` cria um identificador único para cada funcionário. O tipo `INT` indica que o valor será numérico, `AUTO_INCREMENT` faz com que o número seja gerado automaticamente a cada novo cadastro, e `PRIMARY KEY` define essa coluna como chave primária da tabela, garantindo que não existam IDs repetidos. A coluna `nome VARCHAR(100) NOT NULL` armazena o nome do funcionário. O tipo `VARCHAR(100)` permite salvar textos de até 100 caracteres e `NOT NULL` indica que o campo é obrigatório. A coluna `funcao VARCHAR(100) NOT NULL` funciona da mesma forma, armazenando a função ou cargo do funcionário, como gerente, analista ou
```

programador. Já a coluna `salario DECIMAL(10,2) NOT NULL` armazena o salário do funcionário. O tipo `DECIMAL(10,2)` é utilizado para valores monetários, permitindo armazenar números com duas casas decimais, enquanto `NOT NULL` obriga o preenchimento desse campo. Por fim, a coluna `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP` registra automaticamente a data e hora em que o funcionário foi cadastrado. O tipo `TIMESTAMP` armazena informações de data e horário, e `DEFAULT CURRENT_TIMESTAMP` faz com que o MySQL preencha automaticamente esse campo com o horário atual no momento da inserção do registro.

De forma geral, esse script SQL cria toda a base necessária para o funcionamento do sistema CRUD, preparando o banco de dados e a tabela que serão utilizados pelo backend para armazenar, consultar, atualizar e excluir informações dos funcionários cadastrados na aplicação.

Estrutura do Projeto



Index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CRUD</title>
  <link rel="stylesheet" href="style.css">
  <link href='https://unpkg.com/boxicons@2.1.1/css/boxicons.min.css'
rel='stylesheet'>
</head>

<body>
  <div class="container">
    <div class="header">
      <span>Cadastro de Funcionários</span>
      <button onclick="openModal()" id="new">Incluir</i></button>
    </div>

    <div class="divTable">
      <table>
        <thead>
          <tr>
            <th>Nome</th>
            <th>Função</th>
            <th>Salário</th>
            <th class="acao">Editar</th>
            <th class="acao">Excluir</th>
          </tr>
        </thead>
        <tbody>
        </tbody>
      </table>
    </div>

    <div class="modal-container">
      <div class="modal">
        <form>
          <label for="m-nome">Nome</label>
          <input id="m-nome" type="text" required />

          <label for="m-funcao">Função</label>
          <input id="m-funcao" type="text" required />
```

```
          <label for="m-salario">Salário</label>
          <input id="m-salario" type="number" required />
          <button id="btnSalvar">Salvar</button>
        </form>
      </div>
    </div>

  </div>
  <script src="script.js"></script>
</body>

</html>
```

Esse código HTML cria a estrutura de uma aplicação CRUD para cadastro de funcionários. CRUD é um termo utilizado para representar as operações básicas de um sistema: criar, visualizar, atualizar e excluir dados. O documento começa com a declaração `<!DOCTYPE html>`, indicando que a página utiliza HTML5. A tag `<html lang="en">` define o idioma principal da página como inglês. Dentro da seção `<head>` ficam as configurações principais da aplicação, como a codificação de caracteres UTF-8, que permite exibir corretamente letras acentuadas, e a configuração de responsividade por meio da meta viewport, fazendo com que a página funcione bem em celulares e computadores. O título exibido na aba do navegador é definido pela tag `<title>CRUD</title>`. Também são importados dois arquivos externos: o arquivo `style.css`, responsável pela aparência visual da página, e a biblioteca Boxicons, utilizada para adicionar ícones ao sistema.

No corpo da página, dentro da tag `<body>`, existe uma `<div>` com a classe `container`, que funciona como a estrutura principal da aplicação. Dentro dela há uma seção chamada `header`, que exibe o título “Cadastro de Funcionários” e um botão chamado “Incluir”. Esse botão possui o atributo `onclick="openModal()"`, que indica que, ao ser clicado, executará a função `openModal()` definida no JavaScript. Essa função normalmente serve para abrir uma janela modal de cadastro. Logo abaixo existe uma `<div>` chamada `divTable`, responsável por armazenar uma tabela HTML. Essa tabela contém o cabeçalho com cinco colunas: Nome, Função, Salário, Editar e Excluir. O corpo da tabela, representado pela tag `<tbody>`, está vazio porque os dados serão

adicionados dinamicamente através do JavaScript quando os funcionários forem cadastrados.

A seguir aparece a estrutura do modal, criada pela `<div class="modal-container">`. O modal funciona como uma janela que surge sobre a tela para permitir o cadastro ou edição de funcionários. Dentro dele existe um formulário contendo três campos de entrada: nome, função e salário. Cada campo possui um atributo `required`, obrigando o usuário a preencher as informações antes de salvar. Os inputs também possuem IDs específicos (`m-nome`, `m-funcao` e `m-salario`) para que o JavaScript consiga acessar e manipular os valores digitados pelo usuário. Ao final do formulário existe um botão com o ID `btnSalvar`, responsável por salvar os dados preenchidos. Normalmente, ao clicar nesse botão, o JavaScript captura as informações, armazena os dados e atualiza a tabela na tela.

Por fim, a tag `<script src="script.js"></script>` importa o arquivo JavaScript responsável pela lógica do sistema. É nesse arquivo que ficam as funções para abrir e fechar o modal, adicionar novos funcionários, editar registros existentes, excluir dados e atualizar a tabela dinamicamente. Assim, o HTML fornece apenas a estrutura da aplicação, enquanto o CSS define o estilo visual e o JavaScript controla toda a funcionalidade do CRUD.

Aula 26 de maio de 2026

Script,js

```
const modal = document.querySelector('.modal-container')
const tbody = document.querySelector('tbody')
const sNome = document.querySelector('#m-nome')
const sFuncao = document.querySelector('#m-funcao')
const sSalario = document.querySelector('#m-salario')
const btnSalvar = document.querySelector('#btnSalvar')

let itens
let id

function openModal(edit = false, index = 0) {
  modal.classList.add('active')

  modal.onclick = e => {
    if (e.target.className.indexOf('modal-container') !== -1) {
      modal.classList.remove('active')
    }
  }

  if (edit) {
    sNome.value = itens[index].nome
    sFuncao.value = itens[index].funcao
    sSalario.value = itens[index].salario
    id = index
  } else {
    sNome.value = ''
    sFuncao.value = ''
    sSalario.value = ''
  }

}

function editItem(index) {
  id = itens[index].id; // Armazena ID para update
  openModal(true, index);
}

async function deleteItem(index) {
  if (!itens[index]?.id) return;
  try {
    await fetch(`${API_BASE}/${itens[index].id}`, { method: 'DELETE' });
    loadItens();
  } catch (error) {
    console.error('Erro ao deletar:', error);
  }
}
```

Aula 26 de maio de 2026

```
function insertItem(item, index) {
  let tr = document.createElement('tr')

  tr.innerHTML = `
    <td>${item.nome}</td>
    <td>${item.funcao}</td>
    <td>R$ ${item.salario}</td>
    <td class="acao">
      <button onclick="editItem(${index})"><i class='bx bx-edit'
></i></button>
    </td>
    <td class="acao">
      <button onclick="deleteItem(${index})"><i class='bx bx-
trash'></i></button>
    </td>
  `
  tbody.appendChild(tr)
}

btnSalvar.onclick = async e => {

  if (sNome.value == '' || sFuncao.value == '' || sSalario.value == '') {
    return
  }

  e.preventDefault();

  try {
    if (id !== undefined && itens[id]?.id) {
      // UPDATE
      await fetch(`${API_BASE}/${itens[id].id}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          nome: sNome.value,
          funcao: sFuncao.value,
          salario: sSalario.value
        })
      });
    } else {
      // CREATE
      await fetch(API_BASE, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          nome: sNome.value,
          funcao: sFuncao.value,
          salario: sSalario.value
        })
```

```
      });
    }
    modal.classList.remove('active');
    loadItens();
    id = undefined;
  } catch (error) {
    console.error('Erro ao salvar:', error);
    alert('Erro ao salvar. Verifique se servidor está rodando.');
  }
}

async function loadItens() {
  itens = await getItensBD();
  tbody.innerHTML = '';
  itens.forEach((item, index) => {
    insertItem(item, index);
  });
}

// Carrega ao iniciar
loadItens();

const API_BASE = 'http://localhost:3000/api/funcionarios';

const getItensBD = async () => {
  try {
    const response = await fetch(API_BASE);
    if (!response.ok) throw new Error('Erro ao carregar');
    return await response.json();
  } catch (error) {
    console.error('Erro API:', error);
    return [];
  }
};

const setItensBD = () => {}; // Não mais necessário

loadItens()
```

Esse código JavaScript é responsável por controlar toda a lógica do sistema CRUD de funcionários. Ele faz a comunicação entre a interface HTML e a API do backend, permitindo cadastrar, listar, editar e excluir funcionários dinamicamente. No início do código são selecionados vários elementos da página usando `document.querySelector()`. A constante `modal` armazena a janela modal utilizada

para cadastro e edição, enquanto `tbody` representa o corpo da tabela onde os funcionários serão exibidos. As constantes `sNome`, `sFuncao` e `sSalario` armazenam os campos do formulário, e `btnSalvar` representa o botão de salvar.

Depois são declaradas duas variáveis globais: `itens`, que armazenará a lista de funcionários recebidos da API, e `id`, utilizado para controlar qual funcionário está sendo editado. Em seguida é criada a função `openModal(edit = false, index = 0)`, responsável por abrir o modal. Quando executada, ela adiciona a classe `active` ao modal, tornando-o visível na tela. Dentro dessa função também existe um evento de clique que fecha o modal quando o usuário clica fora da caixa principal. Se o parâmetro `edit` for verdadeiro, significa que o usuário está editando um funcionário existente. Nesse caso, os campos do formulário recebem os dados do funcionário selecionado através do array `itens`. Caso contrário, os campos são limpos para um novo cadastro.

A função `editItem(index)` é chamada quando o usuário clica no botão de editar de uma linha da tabela. Ela armazena o ID do funcionário selecionado e chama a função `openModal()` em modo de edição. Já a função `deleteItem(index)` é responsável por excluir funcionários. Ela verifica se o item possui um ID válido e envia uma requisição HTTP do tipo DELETE para a API usando `fetch()`. Após a exclusão, a tabela é atualizada chamando novamente `loadItens()`.

A função `insertItem(item, index)` cria dinamicamente uma linha da tabela usando JavaScript. Ela cria um elemento `<tr>` e insere as informações do funcionário, como nome, função e salário. Também adiciona dois botões: um para editar e outro para excluir. Esses botões executam as funções `editItem()` e `deleteItem()` passando o índice do funcionário selecionado.

O evento principal do sistema está no botão salvar, através de `btnSalvar.onclick`. Quando o botão é clicado, o código primeiro verifica se os campos estão preenchidos. Caso algum esteja vazio, a função é interrompida. Depois o `e.preventDefault()` impede o comportamento padrão do formulário, evitando o recarregamento da página. Em seguida o sistema verifica se existe um ID definido. Se existir, significa que um funcionário está sendo editado, então é enviada uma requisição HTTP PUT para atualizar os dados no backend. Caso contrário, o sistema realiza um cadastro novo

usando uma requisição POST. Em ambos os casos, os dados são enviados em formato JSON. Após salvar, o modal é fechado, a tabela é recarregada e o ID é redefinido. Caso aconteça algum erro durante a comunicação com a API, o sistema exibe uma mensagem no console e um alerta para o usuário.

A função `loadItens()` é responsável por carregar os funcionários cadastrados. Ela chama a função `getItensBD()`, que faz uma requisição GET para a API e retorna os dados em formato JSON. Depois a tabela é limpa e os funcionários são inseridos novamente usando a função `insertItem()`. A função `loadItens()` é executada quando a página inicia, garantindo que os dados apareçam automaticamente na tela.

A constante `API_BASE` define o endereço da API que está sendo utilizada no backend, no caso `http://localhost:3000/api/funcionarios`. A função `getItensBD()` utiliza `fetch()` para buscar os dados da API. Caso a resposta seja bem-sucedida, ela converte os dados para JSON e os retorna. Se ocorrer algum erro, o sistema mostra uma mensagem no console e retorna um array vazio. Por fim, existe a função `setItensBD()`, que não é mais necessária porque agora os dados são manipulados diretamente pela API em vez de serem armazenados localmente.

De forma geral, esse JavaScript transforma a página HTML em um CRUD funcional conectado a um backend. Ele permite abrir formulários, cadastrar funcionários, editar informações existentes, excluir registros e atualizar automaticamente a tabela utilizando requisições HTTP para uma API REST.

Aula 26 de maio de 2026

package,json

```
{
  "name": "backend",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "teste",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.6",
    "express": "^5.2.1",
    "mysql2": "^3.22.2"
  },
  "repository": {
    "type": "git",
    "url": "exit"
  },
  "description": ""
}
```

Esse código representa o arquivo `package.json` de um projeto Node.js. O `package.json` é um arquivo fundamental em aplicações JavaScript que utilizam Node.js, pois ele funciona como uma espécie de configuração geral do projeto. Nele ficam armazenadas informações importantes, como nome do projeto, versão, scripts de execução e dependências utilizadas pela aplicação.

A propriedade `"name": "backend"` define o nome do projeto. Nesse caso, o sistema foi chamado de “backend”, indicando que ele representa a parte do servidor da aplicação CRUD. A propriedade `"version": "1.0.0"` informa a versão atual do projeto, permitindo controlar atualizações e mudanças futuras. Já `"main": "server.js"` define qual arquivo será considerado o ponto de entrada principal da aplicação. Isso significa que, ao iniciar o sistema, o Node.js executará o arquivo `server.js`.

Dentro da seção `"scripts"` ficam os comandos automatizados que podem ser executados pelo npm. O script `"start": "node server.js"` permite iniciar o servidor usando o comando `npm start`. Quando esse comando é executado no terminal, o Node.js roda automaticamente o arquivo `server.js`. Já o script `"test"` é apenas um padrão criado automaticamente pelo npm quando o projeto é inicializado. Ele não possui testes configurados e apenas exibe uma mensagem de erro.

As propriedades `"keywords"`, `"author"` e `"license"` armazenam informações adicionais sobre o projeto. O campo `"author": "teste"` indica o autor da aplicação, enquanto `"license": "ISC"` define o tipo de licença utilizada. O campo `"keywords"` está vazio, mas normalmente é usado para adicionar palavras-chave relacionadas ao projeto.

A seção mais importante do arquivo é `"dependencies"`, pois ela define as bibliotecas externas necessárias para o funcionamento da aplicação. A dependência `"express"` é um framework muito utilizado no Node.js para criar servidores e APIs REST de forma simples e organizada. Nesse projeto, ele provavelmente será utilizado para criar as rotas do CRUD de funcionários. A biblioteca `"cors"` permite liberar o acesso da API para aplicações frontend executadas em domínios diferentes, evitando erros de segurança relacionados ao navegador. Já `"mysql2"` é uma biblioteca responsável por conectar o Node.js ao banco de dados MySQL, permitindo realizar operações como inserir, listar, atualizar e excluir funcionários diretamente no banco de dados.

A seção `"repository"` contém informações sobre o repositório Git do projeto. O campo `"type": "git"` indica que o sistema utiliza Git para controle de versão, enquanto `"url": "exit"` normalmente deveria armazenar o endereço do repositório online, mas nesse caso parece ter sido preenchido apenas como teste.

Por fim, o campo `"description"` serve para adicionar uma descrição resumida do projeto, mas está vazio. Em resumo, esse arquivo `package.json` organiza e gerencia toda a estrutura do backend da aplicação, definindo as bibliotecas necessárias, o ponto de entrada do servidor e os comandos usados para executar o sistema.

Serve.js

```
const express = require('express');
const mysql = require('mysql2/promise');
const cors = require('cors');
const app = express();

app.use(cors());
app.use(express.json());

const dbConfig = {
    host: 'localhost',
    user: 'root',
    password: '',
    database: 'crud_funcionarios'
};

const db = mysql.createPool(dbConfig);

// GET /api/funcionarios
app.get('/api/funcionarios', async (req, res) => {
    try {
        const [rows] = await db.execute('SELECT * FROM funcionarios
ORDER BY id DESC');
        res.json(rows);
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});

// POST /api/funcionarios
app.post('/api/funcionarios', async (req, res) => {
    try {
        const { nome, funcao, salario } = req.body;
        if (!nome || !funcao || !salario) {
            return res.status(400).json({ error: 'Campos obrigatórios'
});
        }
        const [result] = await db.execute(
            'INSERT INTO funcionarios (nome, funcao, salario) VALUES
(?, ?, ?)',
            [nome, funcao, parseFloat(salario)]
        );
        res.json({ success: true, id: result.insertId });
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});

// PUT /api/funcionarios/:id
app.put('/api/funcionarios/:id', async (req, res) => {
```

```
    try {
        const { nome, funcao, salario } = req.body;
        const id = req.params.id;
        await db.execute(
            'UPDATE funcionarios SET nome = ?, funcao = ?, salario = ?
WHERE id = ?',
            [nome, funcao, parseFloat(salario), id]
        );
        res.json({ success: true });
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});

// DELETE /api/funcionarios/:id
app.delete('/api/funcionarios/:id', async (req, res) => {
    try {
        const id = req.params.id;
        await db.execute('DELETE FROM funcionarios WHERE id = ?',
[id]);
        res.json({ success: true });
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});

const PORT = 3000;
app.listen(PORT, () => {
    console.log(`🚀 Servidor rodando em http://localhost:${PORT}`);
    console.log('Endpoints: /api/funcionarios
(GET/POST/PUT/:id/DELETE/:id)');
});
```

Esse código representa o backend de uma aplicação CRUD de funcionários desenvolvido com Node.js, Express e MySQL. Ele é responsável por criar uma API REST capaz de receber requisições do frontend, processar dados e armazenar informações no banco de dados. No início do código são importadas três bibliotecas principais utilizando o comando `require()`. A biblioteca `express` é utilizada para criar o servidor e gerenciar as rotas da aplicação. A biblioteca `mysql2/promise` permite realizar conexões com o banco de dados MySQL utilizando programação assíncrona

com `async/await`, facilitando o tratamento das operações de banco de dados. Já a biblioteca `cors` é utilizada para liberar o acesso da API para aplicações frontend executadas em outro domínio ou porta, evitando bloqueios de segurança do navegador.

Logo depois, a constante `app` recebe a execução do Express através de `express()`, criando a aplicação principal do servidor. Em seguida são utilizados dois middlewares importantes. O primeiro, `app.use(cors())`, habilita o compartilhamento de recursos entre diferentes origens, permitindo que o frontend consiga consumir a API sem erros de CORS. O segundo, `app.use(express.json())`, permite que o servidor interprete dados enviados em formato JSON no corpo das requisições HTTP, algo essencial para operações de cadastro e atualização de funcionários.

O objeto `dbConfig` armazena as informações necessárias para conectar o sistema ao banco de dados MySQL. Nele estão definidos o host do banco (`localhost`), o usuário (`root`), a senha e o nome do banco de dados (`crud_funcionarios`). Em seguida, é criado um pool de conexões através de `mysql.createPool(dbConfig)`. O pool melhora o desempenho da aplicação porque reutiliza conexões já abertas com o banco de dados, tornando as consultas mais rápidas e eficientes.

A primeira rota criada no sistema é:

```
app.get('/api/funcionarios')
```

Essa rota responde às requisições HTTP GET e é responsável por listar todos os funcionários cadastrados no banco de dados. Dentro dela é executado o comando SQL:

```
SELECT * FROM funcionarios ORDER BY id DESC
```

Esse comando busca todos os registros da tabela `funcionarios` e os organiza em ordem decrescente pelo ID, mostrando primeiro os registros mais recentes. O resultado da consulta é armazenado em `rows` e enviado ao frontend em formato JSON usando `res.json(rows)`. Caso aconteça algum erro durante a consulta, o sistema retorna uma resposta com status 500 e envia a mensagem de erro.

A segunda rota é:

```
app.post('/api/funcionarios')
```

Ela é responsável por cadastrar novos funcionários. Quando o frontend envia uma requisição POST, os dados são recebidos através de `req.body`. O sistema utiliza desestruturação para capturar os valores `nome`, `funcao` e `salario`. Depois é realizada uma validação para verificar se todos os campos foram preenchidos. Caso algum esteja vazio, o backend retorna um erro 400 informando que os campos são obrigatórios. Se os dados forem válidos, é executado um comando SQL INSERT:

```
INSERT INTO funcionarios (nome, funcao, salario)
VALUES (?, ?, ?)
```

Os símbolos `?` representam parâmetros preparados, que ajudam a proteger a aplicação contra ataques de SQL Injection. Os dados recebidos são enviados como parâmetros da consulta e o salário é convertido para número decimal usando `parseFloat()`. Após inserir o funcionário no banco, o servidor retorna um JSON indicando sucesso e também o ID do registro criado.

A próxima rota é:

```
app.put('/api/funcionarios/:id')
```

Essa rota é utilizada para atualizar os dados de um funcionário existente. O `:id` na URL representa um parâmetro dinâmico, permitindo identificar qual funcionário será atualizado. O ID é capturado através de `req.params.id`. Em seguida, o backend recebe os novos dados enviados pelo frontend e executa o comando SQL:

```
UPDATE funcionarios
SET nome = ?, funcao = ?, salario = ?
WHERE id = ?
```

Esse comando atualiza as informações do funcionário correspondente ao ID informado. Após concluir a atualização, o servidor retorna um JSON informando que a operação foi realizada com sucesso.

Depois disso existe a rota:

```
app.delete('/api/funcionarios/:id')
```

Essa rota é responsável pela exclusão de funcionários do banco de dados. O sistema captura o ID enviado na URL e executa o comando SQL:

```
DELETE FROM funcionarios WHERE id = ?
```

Esse comando remove o registro correspondente ao ID informado. Após excluir o funcionário, o backend retorna uma resposta JSON indicando sucesso.

Na parte final do código é definida a porta do servidor:

```
const PORT = 3000;
```

Logo depois, o método `app.listen()` inicia o servidor na porta 3000. Quando o servidor é iniciado corretamente, duas mensagens aparecem no console informando que o backend está funcionando e mostrando os endpoints disponíveis da API. Esses endpoints permitem realizar todas as operações CRUD: listar funcionários com GET, cadastrar com POST, atualizar com PUT e excluir com DELETE.

De forma geral, esse código implementa uma API REST completa para gerenciamento de funcionários. Ele conecta o frontend ao banco de dados MySQL, processa requisições HTTP e executa operações de cadastro, consulta, atualização e exclusão de registros utilizando Express e MySQL no ambiente Node.js.

style.css

```css
@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@200;300;400;500;700&family=Roboto:wght@100;300;400;500;700;900&family=Source+Sans+Pro:wght@200;300;400;600;700;900&display=swap');

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: 'Poppins', sans-serif;
}
```

Aula 26 de maio de 2026

```css
body {
  width: 100vw;
  height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  background-color: rgba(0,0,0,0.1);
}

button {
  cursor: pointer;
}

.container {
  width: 90%;
  height: 80%;
  border-radius: 10px;
  background: white;
}

.header {
  min-height: 70px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin: auto 12px;
}

.header span {
  font-weight: 900;
  font-size: 20px;
  word-break: break-all;
}

#new {
  font-size: 16px;
  padding: 8px;
  border-radius: 5px;
  border: none;
  color: white;
  background-color: rgb(57, 57, 226);
}

.divTable {
  padding: 10px;
  width: auto;
  height:inherit;
  overflow:auto;
}
```

```css
.divTable::-webkit-scrollbar {
  width: 12px;
  background-color: whitesmoke;
}

.divTable::-webkit-scrollbar-thumb {
  border-radius: 10px;
  background-color: darkgray;
}

table {
  width: 100%;
  border-spacing: 10px;
  word-break: break-all;
  border-collapse: collapse;
}

thead {
  background-color: whitesmoke;
}

tr {
  border-bottom: 1px solid rgb(238, 235, 235)!important;
}

tbody tr td {
  vertical-align: text-top;
  padding: 6px;
  max-width: 50px;
}

thead tr th {
  padding: 5px;
  text-align: start;
  margin-bottom: 50px;
}

tbody tr {
  margin-bottom: 50px;
}

thead tr th.acao {
  width: 100px!important;
  text-align: center;
}

tbody tr td.acao {
  text-align: center;
```

```
}

@media (max-width: 700px) {
  body {
    font-size: 10px;
  }

  .header span {
    font-size: 15px;
  }

  #new {
    padding: 5px;
    font-size: 10px;
  }

  thead tr th.acao {
    width: auto!important;
  }

  td button i {
    font-size: 20px!important;
  }

  td button i:first-child {
    margin-right: 0;
  }

  .modal {
    width: 90%!important;
  }

  .modal label {
    font-size: 12px!important;
  }
}

.modal-container {
  width: 100vw;
  height: 100vh;
  position: fixed;
  top: 0;
  left: 0;
  background-color: rgba(0, 0, 0, 0.5);
  display: none;
  z-index: 999;
  align-items: center;
  justify-content: center;
}
```

```css
.modal {
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 40px;
  background-color: white;
  border-radius: 10px;
  width: 50%;
}

.modal label {
  font-size: 14px;
  width: 100%;
}

.modal input {
  width: 100%;
  outline: none;
  padding: 5px 10px;
  width: 100%;
  margin-bottom: 20px;
  border-top: none;
  border-left: none;
  border-right: none;
}

.modal button {
  width: 100%;
  margin: 10px auto;
  outline: none;
  border-radius: 20px;
  padding: 5px 10px;
  width: 100%;
  border: none;
  background-color: rgb(57, 57, 226);
  color: white;
}

.active {
  display: flex;
}

.active .modal {
  animation: modal .4s;
}

@keyframes modal {
  from {
```

```
    opacity: 0;
    transform: translate3d(0, -60px, 0);
  }
  to {
    opacity: 1;
    transform: translate3d(0, 0, 0);
  }
}

td button {
  border: none;
  outline: none;
  background: transparent;
}

td button i {
  font-size: 25px;
}

td button i:first-child {
  margin-right: 10px;
}
```

Esse código CSS é responsável por definir toda a aparência visual da aplicação CRUD de funcionários. Ele controla o layout da página, cores, tamanhos, posicionamento dos elementos, responsividade e animações da interface. No início do código é utilizado o comando `@import` para importar fontes do Google Fonts, como Poppins, Roboto e Source Sans Pro. Essas fontes são utilizadas para deixar a interface mais moderna e agradável visualmente. Em seguida, o seletor universal `*` aplica configurações gerais para todos os elementos da página, removendo margens e espaçamentos padrões com `margin: 0` e `padding: 0`, além de utilizar `box-sizing: border-box`, que facilita o controle dos tamanhos dos elementos. Também é definida a fonte padrão da aplicação como `Poppins`.

A estilização do `body` faz com que o conteúdo ocupe toda a largura e altura da tela utilizando `100vw` e `100vh`. Além disso, o `display: flex` junto com `align-items: center` e `justify-content: center` centraliza toda a aplicação no meio da tela. O fundo da página recebe uma cor cinza clara utilizando `background-color: rgba(0,0,0,0.1)`. Os botões recebem a propriedade `cursor: pointer`, fazendo com que o cursor do mouse mude para uma mãozinha ao passar sobre eles.

A classe `.container` representa a caixa principal da aplicação. Ela recebe largura de 90%, altura de 80%, fundo branco e bordas arredondadas para criar um visual moderno. Já a classe `.header` organiza o título e o botão “Incluir” utilizando Flexbox

para alinhar os elementos horizontalmente e distribuir espaço entre eles. O texto do cabeçalho recebe peso maior e tamanho aumentado através de `.header span`. O botão com ID `#new` recebe estilizações específicas como fundo azul, texto branco, remoção da borda, espaçamento interno e bordas arredondadas.

A classe `.divTable` controla a área da tabela, adicionando espaçamento interno e permitindo rolagem com `overflow: auto` caso o conteúdo ultrapasse o espaço disponível. Também são personalizadas as barras de rolagem usando os seletores `::-webkit-scrollbar` e `::-webkit-scrollbar-thumb`, alterando largura, cor e arredondamento da barra. A tabela recebe largura total, espaçamento entre células e remoção de bordas duplicadas através de `border-collapse: collapse`. O cabeçalho da tabela (`thead`) recebe fundo cinza claro para destacar os títulos das colunas. As linhas da tabela recebem bordas inferiores suaves, enquanto células e colunas possuem alinhamentos e espaçamentos específicos para melhorar a organização visual.

O código também possui uma media query:

```
@media (max-width: 700px)
```

Essa parte é responsável pela responsividade da aplicação. Quando a largura da tela for menor que 700 pixels, como em celulares, alguns elementos terão seus tamanhos reduzidos para melhorar a visualização. O tamanho das fontes diminui, os botões ficam menores e o modal ocupa mais espaço horizontal na tela, adaptando a interface para dispositivos móveis.

A classe `.modal-container` representa o fundo escurecido exibido quando o modal é aberto. Ela ocupa toda a tela usando `width: 100vw` e `height: 100vh`, possui posição fixa e um fundo semitransparente preto criado com `rgba(0, 0, 0, 0.5)`. Inicialmente o modal fica escondido usando `display: none`. Quando a classe `.active` é adicionada pelo JavaScript, o modal passa a ser exibido usando `display: flex`, centralizando a caixa do formulário na tela.

A classe `.modal` estiliza a janela do formulário. Ela utiliza Flexbox para organizar os elementos em coluna, possui fundo branco, espaçamento interno, largura de 50% e bordas arredondadas. Os campos de entrada recebem largura total, remoção de bordas laterais e superiores, além de espaçamento inferior para separar os campos visualmente. O botão do modal também recebe largura total, fundo azul, texto branco e bordas arredondadas.

O código ainda define uma animação chamada `modal` utilizando `@keyframes`. Essa animação faz o modal surgir suavemente na tela com efeito de fade e movimento vertical. Inicialmente o modal aparece invisível e deslocado para cima, depois vai se tornando visível até alcançar sua posição final.

Por fim, os botões dentro da tabela recebem fundo transparente e remoção de bordas para deixar apenas os ícones visíveis. Os ícones possuem tamanho aumentado para facilitar a interação do usuário. Em resumo, esse CSS é responsável por transformar a estrutura HTML simples em uma interface moderna, organizada, responsiva e

agradável visualmente, controlando desde o layout principal até animações e comportamento visual do modal.